

▼ 3 Algorithms

▼ Introduction

In this chapter we supplement the discussion of algorithms presented in the text with their implementation in Maple. In Section 3.1, we discuss the process of turning step-by-step instructions describing a procedure and pseudocode for a procedure into Maple code. In the second section, we make use of Maple's graphing capabilities to visualize functions related by the big-O notation. And in Section 3.3, we explore average-case complexity of algorithms by considering the performance of a procedure on input.

In this chapter, please keep in mind the difference between an algorithm and its implementation (referred to as a procedure in Maple parlance). An algorithm refers to an approach to solving a particular problem, while a procedure is the implementation of the algorithm within Maple. In this manual, we will also distinguish between complexity and performance. Complexity is a measure of an algorithm and is generally measured by counting basic operations such as comparisons, while performance takes into account additional factors related to the specifics of an implementation and can be measured by recording the time it takes for the procedure to complete. Some of the factors affecting performance may include: choice of data structures and how the system implements those structures, the kinds of loops employed and how those are implemented by the computer language, and various improvements to efficiency handled by the system (for example, many computer languages, when evaluating a boolean expression such as p **and** q , will not bother checking q if p is found to be false thereby decreasing the number of operations that need to be performed).

▼ 3.1 Algorithms

It is impossible to overemphasize the importance and utility of writing either pseudocode or step-by-step instructions for an algorithm before you write the actual code for the procedure. Doing so helps you organize your ideas about how to solve the problem without the rigid constraints of the particular programming language. The textbook serves as an excellent model for you as you learn how to turn mathematical concepts and solutions to problems into algorithms. Those algorithms can then be turned into procedures in any programming language you choose. This manual will help you turn your pseudocode or step-by-step instructions for algorithms into procedures written in Maple.

Section 3.1 of the textbook describes several algorithms, with an emphasis on how you can describe these algorithms using both English descriptions and pseudocode. Here, we will see how to implement several of these algorithms in Maple. In this chapter, it will be especially important for you to have the text alongside you, as we will not reproduce the descriptions of the algorithms from the text.

Find Maximum

The first algorithm we will implement is the algorithm for finding the maximum in an unsorted sequence. This is described in Section 3.1 in the solution to Example 1 as step-by-step instructions and as pseudocode in Algorithm 1. We will build the procedure according to the step-by-step instructions. In order to make the connection between the instructions and the code as clear as possible, we'll begin with a procedure with no statements and successively revise it to show the addition of each step. Be warned that the incomplete versions of the procedure may produce errors.

We begin with the basic elements of the procedure definition without code. Specifically, we include the assignment to the name of the procedure, the input with its type, and, since we will have local variables, the local keyword.

```
> FindMax := proc(L::list(integer))
    local ;

    end proc:
Error, `;` unexpected
```

Note that we've written our procedure definition to accept a list, rather than a "finite sequence of integers" as specified by the text. This is because, in Maple, if you give a sequence as the argument to a procedure, Maple interprets that as multiple arguments and the parameter would be set to the first value only.

Step 1 in the step-by-step instructions is to "set the temporary maximum equal to the first integer" in the list. We declare a local variable to store the temporary maximum and add a statement in the body of the procedure to assign the first integer in the list to this value.

```
> FindMax := proc(L::list(integer))
    local tempMax;
    tempMax := L[1];

    end proc:
```

Step 2, according to Example 1 of the text, is to compare the next integer to the temporary maximum and update the temporary maximum if necessary. This requires an if statement to make the comparison.

```
> FindMax := proc(L::list(integer))
    local tempMax;
    tempMax := L[1];
    if tempMax < L[2] then
        tempMax := L[2];
    end if;

    end proc:
```

(We are intentionally following the step-by-step instructions to the letter, so in step 2, "the next integer" is **L[2]**.)

Step 3 tells us to repeat step 2 for all of the integers in the list. We need to revise our code as follows. The fact that we are supposed to repeat step 2 means that we'll need a loop. And since this loop is supposed to consider all of the elements of the list beyond the first means that we use a for loop over the indices of the list starting at 2. We put the code we wrote for step 2 inside this for loop, since that's the instruction being repeated, and replace the specific index "2" with the loop variable. Finally, the loop variable needs to be added to the **local** list.

```
> FindMax := proc(L::list(integer))
    local tempMax, i;
    tempMax := L[1];
    for i from 2 to nops(L) do
        if tempMax < L[i] then
            tempMax := L[i];
        end if;
    end do;
    end proc:
```

Finally, step 4 tells us that once the loop is completed, the value of the temporary maximum is the

maximum, so we return that value.

```
> FindMax := proc(L::list(integer))
    local tempMax, i;
    tempMax := L[1];
    for i from 2 to nops(L) do
        if tempMax < L[i] then
            tempMax := L[i];
        end if;
    end do;
    return tempMax;
end proc;
> FindMax([3,18,-5,72,6,0]);
```

72

(3.1)

Admittedly, this example may be a bit too simple to warrant such an elaborate process. But it illustrates an essential point: a well-written set of step-by-step instructions describing a procedure can be easily turned into working and correct code. Moreover, for non-trivial algorithms, the two-step process of writing instructions for the procedure and then implementing the procedure based on those instructions typically results in the production of a correct implementation more quickly than attempting the implementation without writing instructions.

Take a moment to compare the procedure above with the pseudocode given in Algorithm 1. You will notice that, in this example, they are extremely similar. This is one of the benefits of pseudocode in comparison to step-by-step instructions. However, step-by-step instructions are often easier to write as a first step towards a working procedure. Step-by-step instructions are also typically easier to understand, especially for novice programmers, as they can more easily accommodate explanation and other information that is out of place in pseudocode.

Binary Search

The second example will be the binary search algorithm, presented as Algorithm 3 in Section 3.1. The previous example showed how you can use step-by-step instructions to build up the procedure. Starting from pseudo-code, writing the final implementation involves translating statements in the pseudocode into legal statements in the programming language and filling in missing details.

As before, we will go through several iterations as we translate the pseudocode in the text to actual Maple code. Initially, we need to make sure that the input is specified in a way appropriate for Maple; declare the local variables that are indicated in the pseudocode; and make basic syntax adjustments, specifically the while loops, if statements, and mathematical expressions such as $\lfloor (i+j)/2 \rfloor$ must be translated into their Maple counterparts. In this case, we'll turn the sequence $a_1, a_2, \dots, a_n$ of integers specified as input in the text into a list **A**.

```
> binarysearch := proc(x::integer,A::list(integer))
    local i, j, m, location;
    i := 1;
    j := n;
    while i < j do
        m := floor((i+j)/2);
        if x > A[m] then
            i := m + 1;
        else
            j := m;
        end if;
    end while;
    return i;
```

```

end do;
if x = A[i] then
    location := i;
else
    location := 0;
end if;
return location;
end proc:

```

It is too much to expect this first version to properly execute.

```

> binarysearch(19, [1,2,3,5,6,7,8,10,12,13,15,16,18,19,20,22]);
Error, (in binarysearch) cannot determine if this expression
is true or false: 1 < n

```

The problem is a simple one to correct. The pseudocode used n as the last index of the sequence of integers, so we need to make that assignment in our code.

```

> binarysearch := proc(x::integer,A::list(integer))
    local n, i, j, m, location;
    n := nops(A);
    i := 1;
    j := n;
    while i < j do
        m := floor((i+j)/2);
        if x > A[m] then
            i := m + 1;
        else
            j := m;
        end if;
    end do;
    if x = A[i] then
        location := i;
    else
        location := 0;
    end if;
    return location;
end proc:

```

Now we try running it again:

```

> binarysearch(19, [1,2,3,5,6,7,8,10,12,13,15,16,18,19,20,22]);
14
(3.2)

```

Observe that the only commands that we added were the declaration of local variables, the assignment of n , and ending the loop and if statements. We also added additional line breaks to be consistent with the style of code used in this manual. Otherwise, the Maple code and pseudocode match very closely. It is, of course, not always quite this straightforward, but well-written pseudocode should contain all the essential elements. Like with proofs, as you become more familiar with pseudocode you will find yourself more comfortable with leaving some details out.

Bubble Sort

As a final example, we will implement the bubble sort, presented in the text as Algorithm 4 of Section 3.1.

For our first attempt at implementing Algorithm 4, we need to: specify the input, declare the local variables (which can, in part, be gleaned from the pseudocode), and correct the syntax for the for loops and if statements.

```

> bubblesort := proc(A::list(realcons))
    local i, j;
    for i from 1 to n - 1 do
        for j from 1 to n - i do
            if A[j] > A[j+1] then
                # interchange A[j] and A[j+1]
            end if;
        end do;
    end do;
end proc:

```

(The realcons type is Maple's type for real numbers.)

The implementation above gets us close to a correct implementation of the bubble sort algorithm, but there are some problems. The simplest to fix is that **n** is not defined. You can correct this in one of two ways. Either declare **n** as a local variable and set it equal to the number of elements of **A**, or replace the two occurrences of **n** with **nops(A)**. There is little difference between these solutions.

```

> bubblesort := proc(A::list(realcons))
    local i, j, n;
    n := nops(A);
    for i from 1 to n - 1 do
        for j from 1 to n - i do
            if A[j] > A[j+1] then
                # interchange A[j] and A[j+1]
            end if;
        end do;
    end do;
end proc:

```

The next problem is the instruction in the pseudocode to interchange a_j with a_{j+1} . Maple contains no interchange command. So we'll either have to make an **interchange** procedure that can be used within the **bubblesort** procedure or flesh out the code to interchange the two list elements within **bubblesort** itself. We take the first approach here so as to preserve as close a connection as possible between our implementation and the pseudocode.

Before creating an **interchange** procedure, however, there is another issue to take into consideration. Namely, in Maple, arguments to a procedure cannot be modified. Instead, we need to copy the parameter to a local variable, both in **bubblesort** and in the **interchange** procedure we are about to write.

Our **interchange** procedure will take two arguments: the list of numbers and the index of the smaller of the two positions to be swapped. It will proceed as follows:

1. Set a temporary variable equal to the first value to be swapped.
2. Set the value of the first position equal to the second value.
3. Set the value of the second position equal to the value stored in the temporary variable.

```

> interchange := proc(L::list, i::posint)
    local M, temp;
    M := L;
    temp := M[i];
    M[i] := M[i+1];
    M[i+1] := temp;
    return M;
end proc:

```

Now we update our implementation by: copying the list to a local name and replacing all the occurrences of the parameter with the local variable; applying the **interchange** procedure; and ending the procedure by explicitly returning the local copy of the list.

```
> bubblesort := proc(A::list(realcons))
    local B, i, j, n;
    B := A;
    n := nops(B);
    for i from 1 to n - 1 do
        for j from 1 to n - i do
            if B[j] > B[j+1] then
                B := interchange(B,j);
            end if;
        end do;
    end do;
    return B;
end proc;
> bubblesort([3,18,-5,72,6,0]);
      [-5, 0, 3, 6, 18, 72]
```

(3.3)

▼ 3.2 The Growth of Functions

In this section we will use Maple to computationally explore the growth of functions. In particular, we will graph functions in order to visually convince ourselves that the big-O relationship is satisfied. We will also see how to use graphs to determine possible witnesses for the constants C and k in the definition of big-O notation. Since, as the textbook mentions, $f(x)$ is $O(g(x))$ if and only if $g(x)$ is $\Omega(f(x))$, the techniques we explore in this section apply also to big-Omega and big-Theta notation.

We begin by considering the function $f(x) = 5x^3 + 4x^2 + 3x + 9$. Theorem 1 from Section 3.2 tells us that this is $O(x^3)$, but we will use this function as an example of how you can use Maple to find values for C and k such that $|f(x)| \leq C|g(x)|$ for all $x \geq k$.

The plot Command

First we'll look at options to the **plot** command that will be useful in this context. Let's start by giving names to the formulas.

```
> f1 := 5*x^3 + 4*x^2 + 3*x + 9;
      f1 := 5x3 + 4x2 + 3x + 9
```

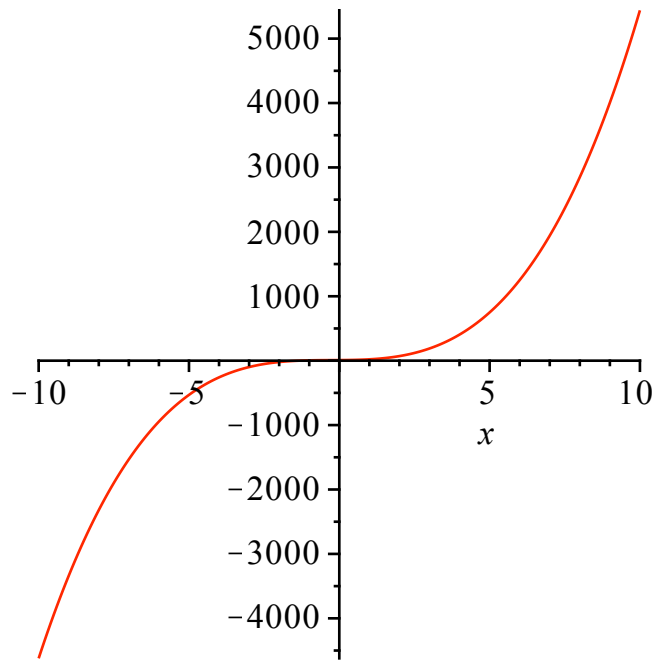
(3.4)

```
> g1 := x^3;
      g1 := x3
```

(3.5)

Graphing $f(x)$ can be as simple as the following.

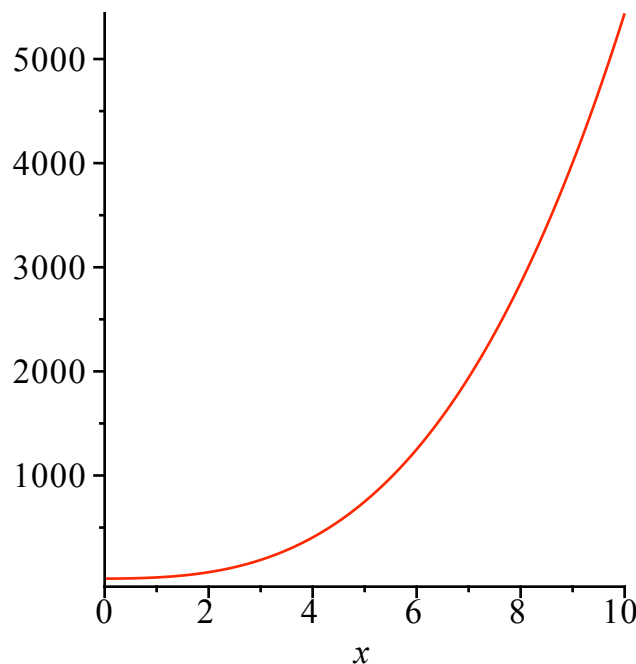
```
> plot(f1,x);
```



The first argument is the function in terms of an independent variable and the second variable is the name of the variable.

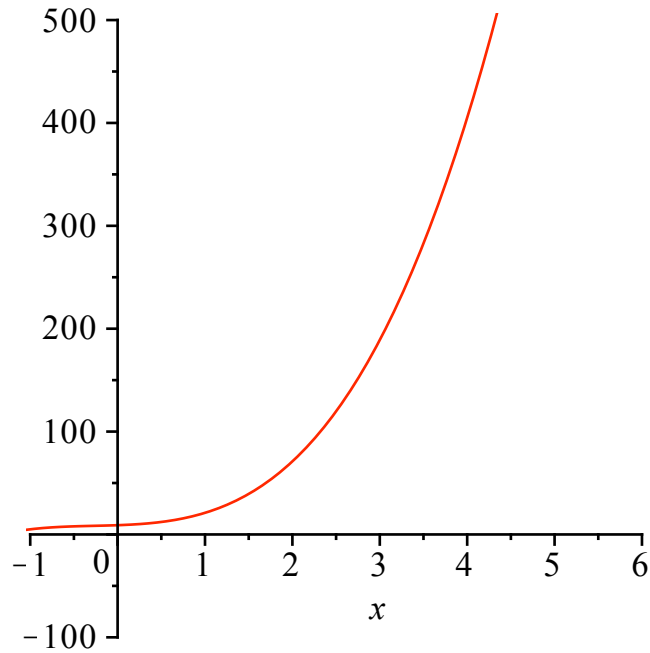
By default, Maple displays the graph with the horizontal axis ranging from -10 to 10. You can specify a different range of x values as follows.

```
> plot(f1,x=0..10);
```



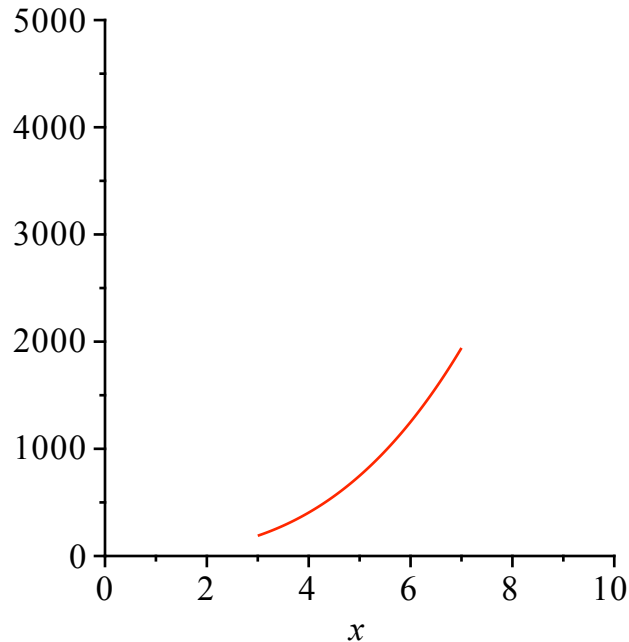
Note that Maple automatically selects the vertical range of the graph. You can control this with the **view** option. You use the view option by setting the keyword **view** equal to a two-element list. The first element of the list consists of the horizontal range to be displayed and second member of the list is the vertical range to be displayed. For example, to display our graph with x ranging from -1 to 6 and y restricted between -100 and 500, you enter the following command.

```
> plot(f1,x,view=[-1..6,-100..500]);
```



It should be observed that, despite a somewhat similar effect, the view option is quite different from setting the range of the independent variable. We can illustrate the difference with the following example.

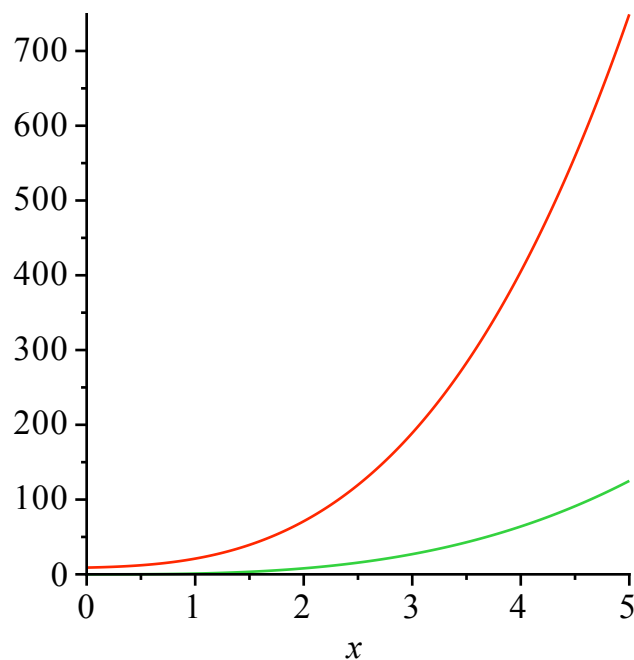
```
> plot(f1,x=3..7,view=[0..10,0..5000]);
```



You see above that the view option is specifying the extent of the graph. On the other hand, setting the range $x=3..7$ is actually a restriction of the domain of the function.

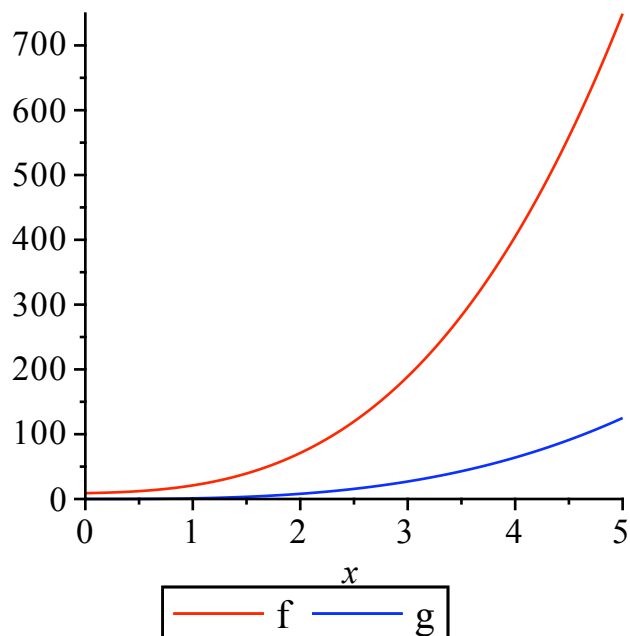
To plot multiple functions in the same graph, we merely issue the plot command with a list of the functions as the first argument.

```
> plot([f1,g1],x=0..5);
```

Note that Maple automatically selects colors for the two functions. You can manually select the colors you want with the **color** option. If you set the **color** keyword equal to a list of [color names](#), the first color is assigned to the first function, the second color to the second function, and so on. You can also create a legend by setting the **legend** keyword equal to a list of strings identifying the functions.

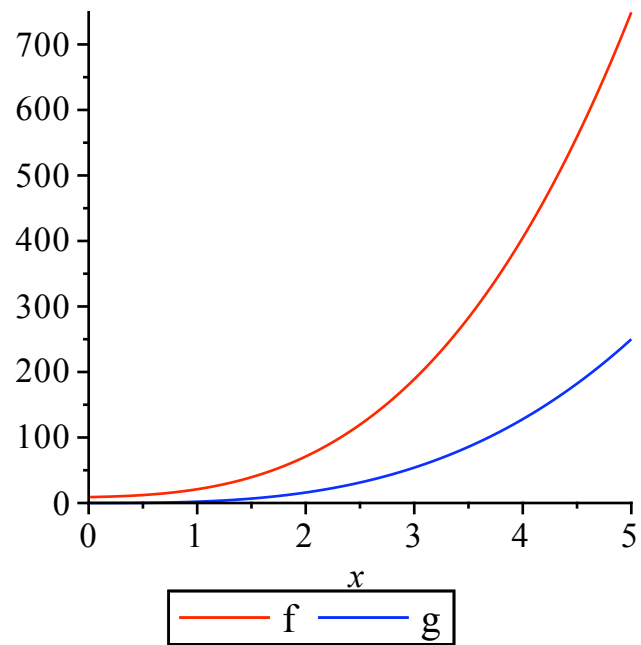
```
> plot([f1,g1],x=0..5,color=[red,blue],legend=["f","g"]);
```



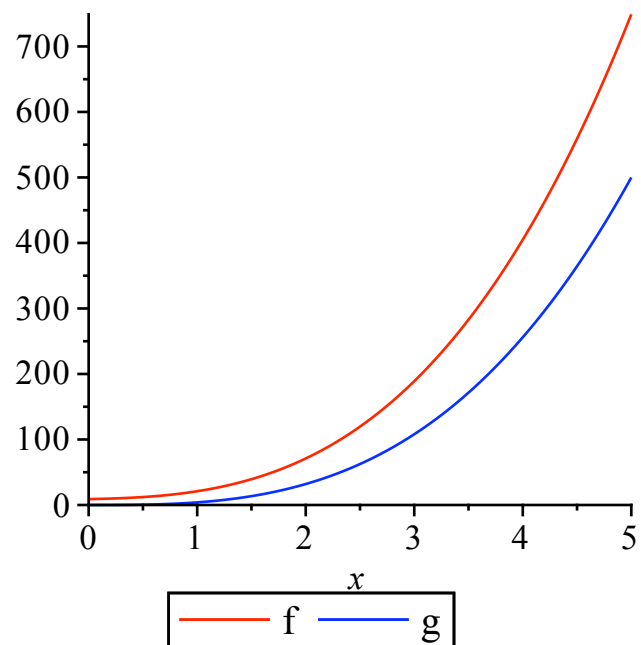
Finding Values for C and k

Now we can start exploring different values of C for which the equation $f(x) \leq Cg(x)$ is satisfied. To do this, we just have to multiply **g1(x)** by different values within the list of functions. We will choose several values until we see a clear crossing.

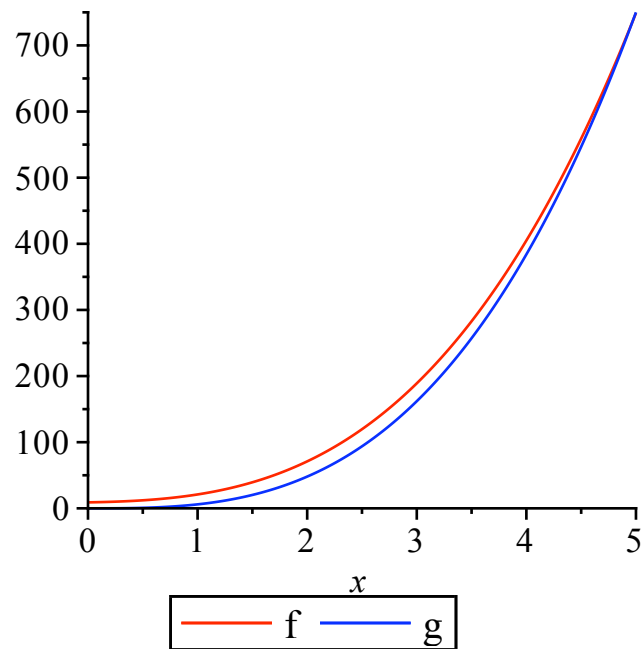
```
> plot([f1,2*g1],x=0..5,color=[red,blue],legend=["f","g"]);
```



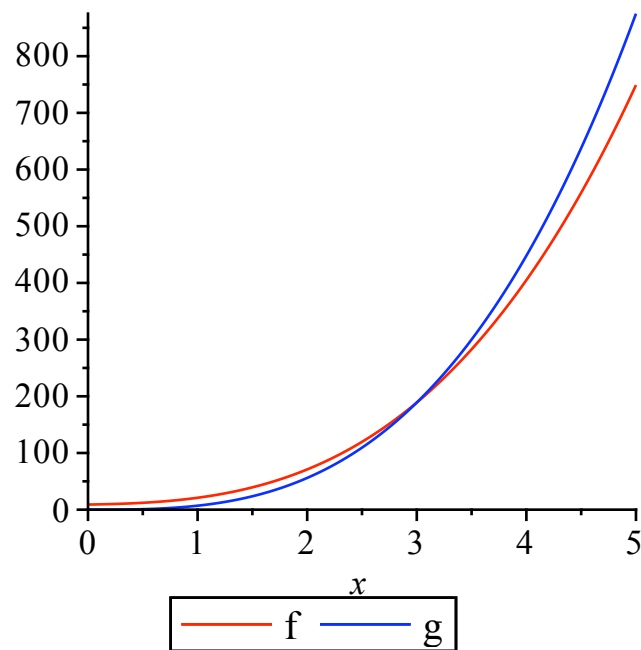
```
> plot([f1,4*g1],x=0..5,color=[red,blue],legend=["f","g"]);
```



```
> plot([f1,6*g1],x=0..5,color=[red,blue],legend=["f","g"]);
```

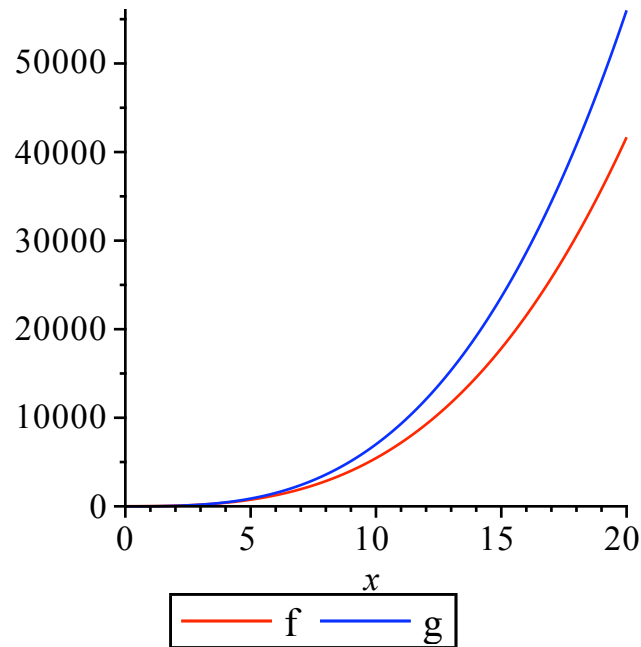


```
> plot([f1,7*g1],x=0..5,color=[red,blue],legend=["f","g"]);
```



By expanding the range of x values, we can obtain a graph that provides fairly convincing evidence that $C = 7$ and $k = 3$ witness for the assertion that $f(x)$ is $O(x^3)$.

```
> plot([f1,7*g1],x=0..20,color=[red,blue],legend=["f","g"]);
```



It is important to note that the graph above is not *proof* that $5x^3 + 4x^2 + 3x + 9$ is $O(x^3)$. A formal proof must follow the model provided by the examples in the text.

A Second Example

As a second example, consider $f(x) = 3x^5 + x^3 \ln(x^2 + 2x + 1)$. We claim that this is $O(x^n)$ for some value of n . We want to first determine the smallest value of n and then find witnesses for C and k . We assign a name for the formula for $f(x)$.

$$\begin{aligned} &> \text{f2} := 3 * x^5 + x^3 * \log(x^2 + 2 * x + 1); \\ &\quad f2 := 3x^5 + x^3 \ln(x^2 + 2x + 1) \end{aligned} \tag{3.6}$$

For $g(x)$, we'll define a functional operator that accepts the exponent as input and returns the formula x^n .

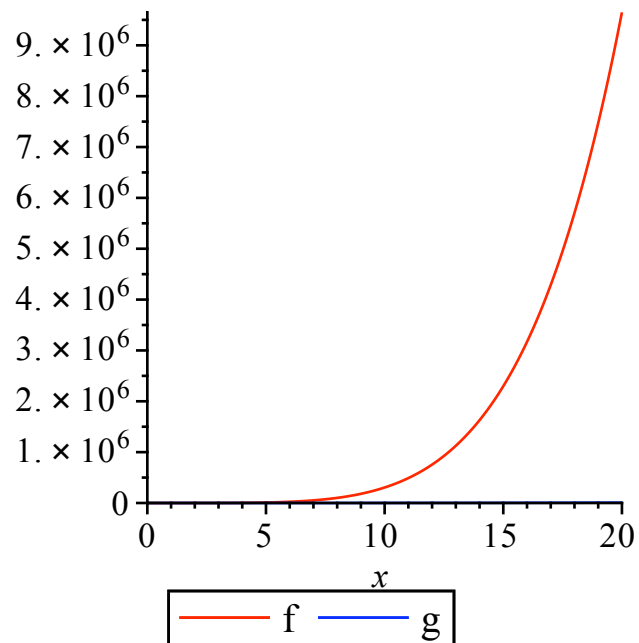
$$\begin{aligned} &> \text{g2} := n \rightarrow x^n; \\ &\quad g2 := n \rightarrow x^n \end{aligned} \tag{3.7}$$

Applying **g2** to 3, for instance, produces the formula x^3 .

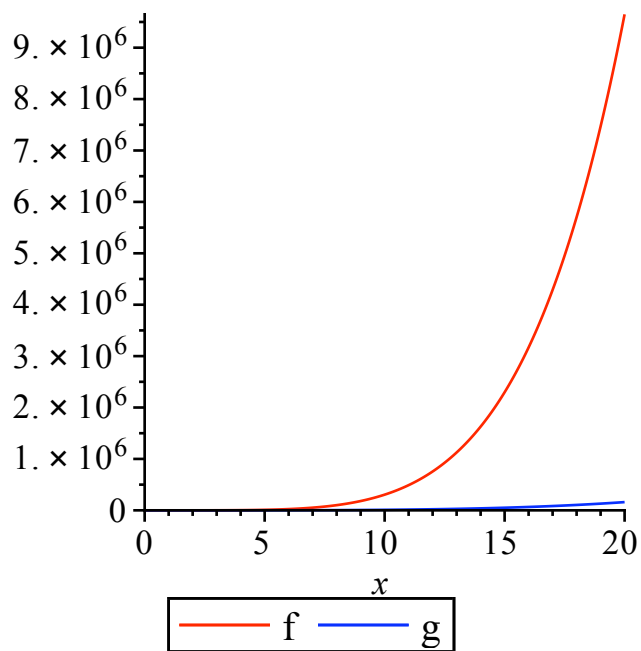
$$\begin{aligned} &> \text{g2}(3); \\ &\quad x^3 \end{aligned} \tag{3.8}$$

Now we can explore the graph of $f(x)$ in comparison to x^n .

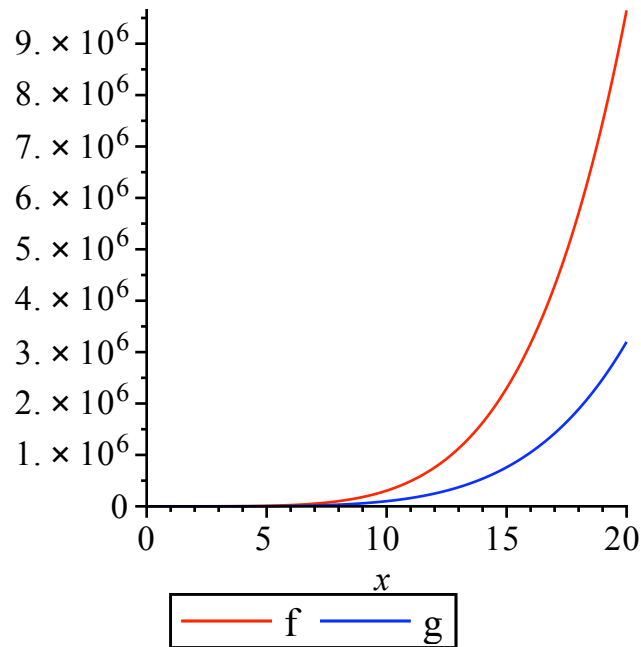
$$> \text{plot}([f2, g2(3)], x=0..20, color=[red, blue], legend=["f", "g"]);$$



```
> plot([f2,g2(4)],x=0..20,color=[red,blue],legend=["f","g"]);
```

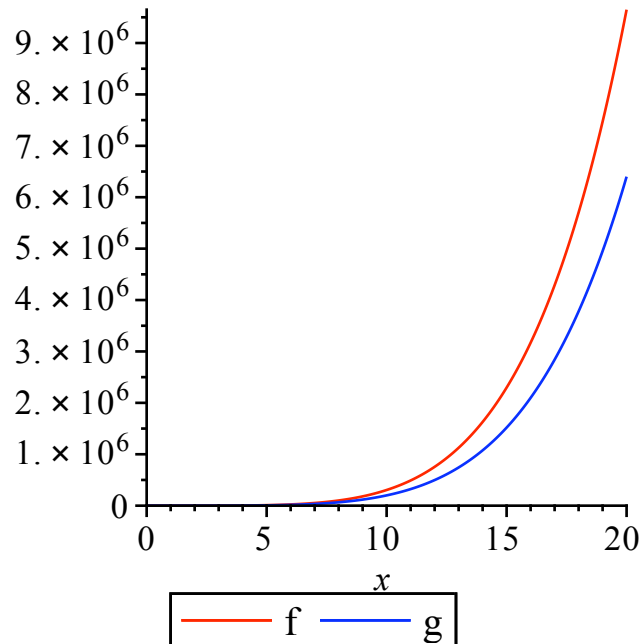


```
> plot([f2,g2(5)],x=0..20,color=[red,blue],legend=["f","g"]);
```

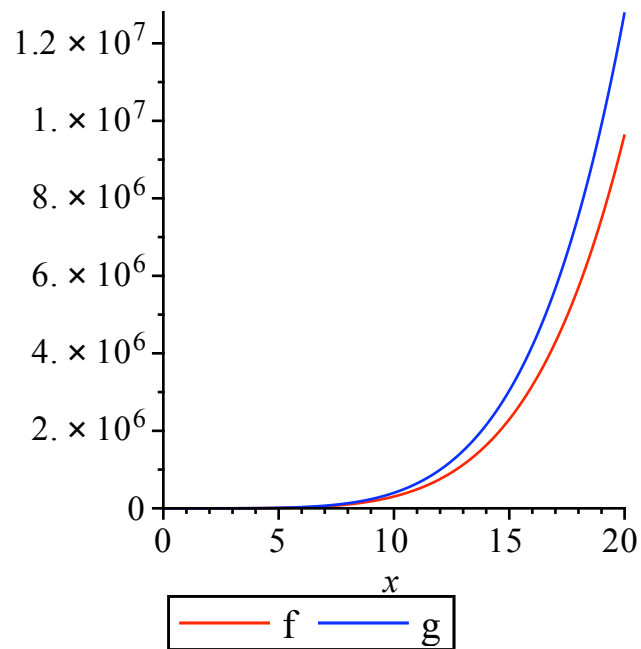


For $n = 3$, x^3 grows so slowly relative to $f(x)$ that it is virtually invisible on the graph. For $n = 4$, the graph of x^4 is clearly visible, but clearly growing at a much slower rate than $f(x)$. For $n = 5$, however, there is some hope that multiplying by a constant may enable x^5 to catch up to $f(x)$. So let's explore some values of C .

```
> plot([f2, 2*g2(5)], x=0..20, color=[red, blue], legend=["f", "g"]);
```

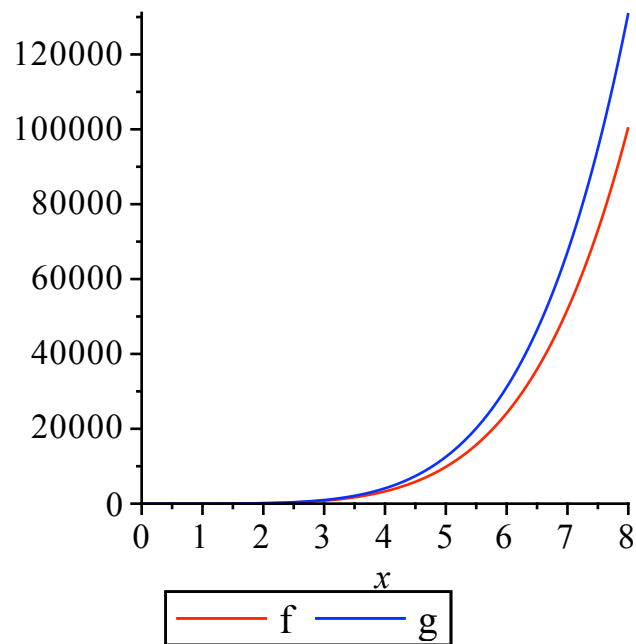


```
> plot([f2, 4*g2(5)], x=0..20, color=[red, blue], legend=["f", "g"]);
```

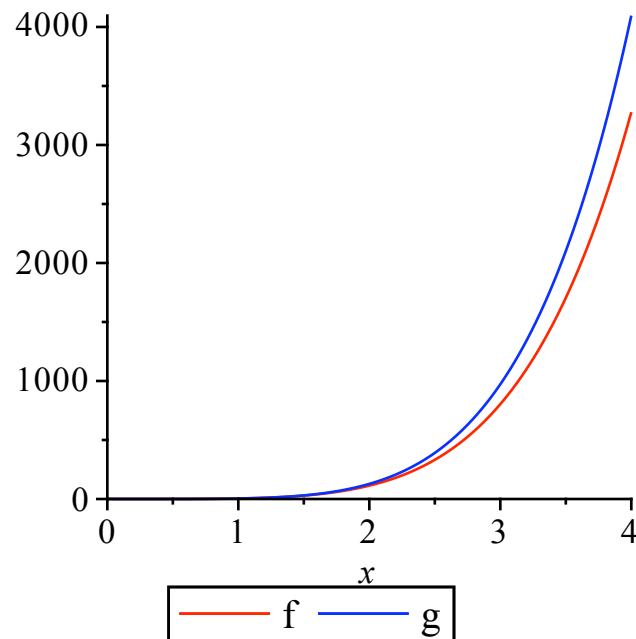


It appears that $C = 4$ is sufficient for $f(x) \leq 4 \cdot x^5$. Let's "zoom in" to see if we can get a potential value for k .

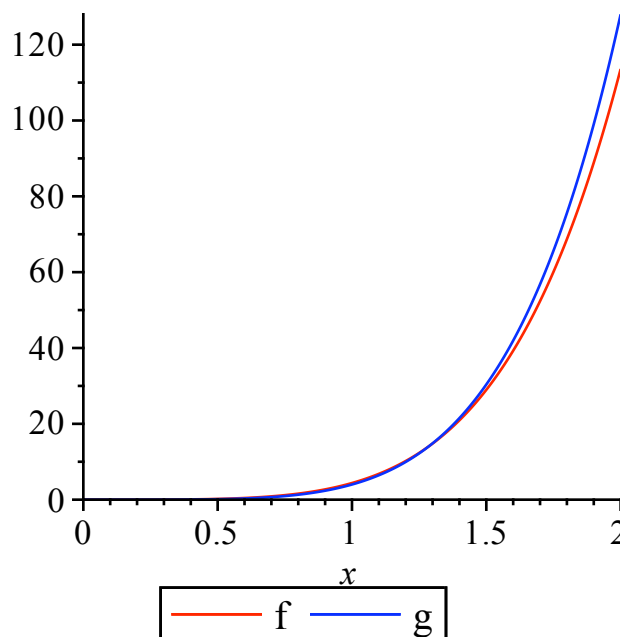
```
> plot([f2, 4*g2(5)], x=0..8, color=[red, blue], legend=["f", "g"]);
```



```
> plot([f2, 4*g2(5)], x=0..4, color=[red, blue], legend=["f", "g"]);
```



```
> plot([f2,4*g2(5)],x=0..2,color=[red,blue],legend=["f","g"]);
```



By successively tightening the range of x values, we can see that $k = 1.5$ appears to be sufficient.

▼ 3.3 Complexity of Algorithms

Section 3.3 of the textbook emphasizes worst-case complexity of algorithms and shows you how the worst-case complexity can be deductively determined. The textbook also mentions average-case complexity and shows how to compute the average-case complexity of the linear search algorithm (Example 4).

Average-case complexity is typically more difficult to analyze deductively, but is still very important. From a practical standpoint, average-case complexity can help differentiate algorithms whose worst-case complexities are of the same order. Also, algorithms that have very poor worst-

case complexity may have reasonable average-case complexity, provided that the "bad" inputs that produce the worst case are rare.

While average-case complexity is difficult to analyze, average-case performance can be computed fairly directly. Recall from the introduction to this chapter that we distinguish complexity of an algorithm from performance of a procedure. In this section we will see how Maple can be used to analyze average-case performance of procedures experimentally. We will use the **bubblesort** procedure developed in Section 3.1 of this manual as an example. Our goal will be to produce a graph displaying the empirically determined average-case time performance of the procedure.

Timing Procedures

First, we review the standard approach to timing procedures in Maple. The **time** command returns the total CPU time that has been used by Maple since the start of the Maple session. By computing this value before running a procedure and again after the procedure has been completed, the difference in the values will be a very good estimate of the time taken to run the procedure. It is not exact because the initial time value must be assigned to a name, and that assignment takes some time.

Here is an example of the use of the **time** command.

```
> st := time():
  FindMax([$1..10000]):
  time() - st;
                                0.009
```

(3.9)

The name **st** stands for "start time." Recall that **[\$1..10000]** produces the list consisting of the integers 1 through 10000. Note that colons are used to suppress all the output except the final command that reports the elapsed time. This is important because if the procedure produces output, the display of the output takes time. We do not want the time it takes for Maple to display the results included as part of the time performance of the procedure.

Average Input

By average-case performance, we mean the average performance of a procedure on a random input selected from all possible inputs of the given size. The particulars of how the random input is selected is a necessary component in the analysis. It is natural to assume that each possible input will appear with the same likelihood as every other, but it is important to recognize that this may not always be the case. It may be that, in the circumstances under which the algorithm is intended to be used, some inputs may appear with relatively higher or lower frequency.

In our test of **bubblesort**, we have no particular application in mind and so will assume that all inputs are equally likely. In order to generate a random input, we will use the **randperm** command from the **combinat** package. Applied to a positive integer n , **randperm** will produce a list of the integers from 1 to n in random order. (We describe the commands of the **combinat** package in more detail in Chapter 6.)

```
> combinat[randperm](20);
[20, 17, 7, 8, 14, 13, 18, 2, 5, 16, 11, 3, 1, 4, 19, 10, 6, 9, 15, 12]
```

(3.10)

We can apply the **bubblesort** algorithm directly to the result and time how long it takes to execute.

```
> st := time():
  bubblesort(combinat[randperm](20)):
  time() - st;
```

0.001

(3.11)

Since we are after average-case performance, we'll want to execute **bubblesort** on some number, say 100, of different random inputs and average the time taken by each execution. To collect the 100 times, we can use a for loop to build a list in which the times are stored.

```
> timings := []:  
  for i from 1 to 100 do  
    st := time():  
    bubblesort(combinat[randperm](20)):  
    et := time() - st:  
    timings := [op(timings), et]:  
  end do:  
> timings;  
[0.001, 0.001, 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0.001, (3.12)  
  0., 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0.013, 0., 0.001, 0.001, 0., 0.001, 0.001, 0.,  
  0.001, 0.001, 0., 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0.,  
  0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0., 0.001, 0.001, 0.001,  
  0., 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0., 0.001, 0.001, 0.001, 0.,  
  0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0., 0.001, 0.001, 0.001,  
  0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0.001, 0.001, 0., 0.001, 0., 0.001, 0.001]
```

(Depending on the speed of your computer, you may need to increase the size of the input list in order to get any non-zero times.)

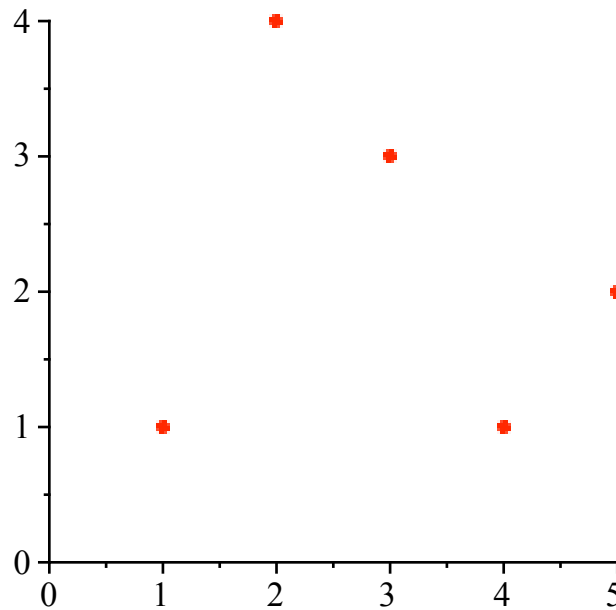
To average the times, we apply the **Mean** command from the **Statistics** package to the list of values.

```
> Statistics[Mean](timings);  
0.0008600000000 (3.13)
```

Graphing the Empirically Calculated Average-Case Complexity

To graph the average time data, we'll use the **plot** command. We saw above how to use plot to graph functions defined by formulas. We can also use plot to draw graphs by giving it specific values for the x and y coordinates. The coordinates must be given as two lists — the first list consisting of the x values and the second list the corresponding y values.

```
> plot([1,2,3,4,5], [1,4,3,1,2], view=[0..5, 0..4], style=point,  
  symbol=solidcircle, symbolsize=15);
```



We have already seen the [view option](#). The option `style=point` causes Maple to display only the points specified in the lists. Omitting `style=point` results in a graph in which the data are connected by straight line segments. The `symbol=solidcircle` determines the symbol used to plot the points. Other options include `asterisk`, `box`, `solidbox`, and `circle`. Finally, `symbolsize=15` increases the size of the dots to 15 points.

We will now write a procedure that produces the two lists required by [plot](#). This procedure will accept no arguments, but will compute the average, over 100 trials, of the time taken to execute `bubblesort` on randomly generated lists of size 10, 20, 30, 40, and 50.

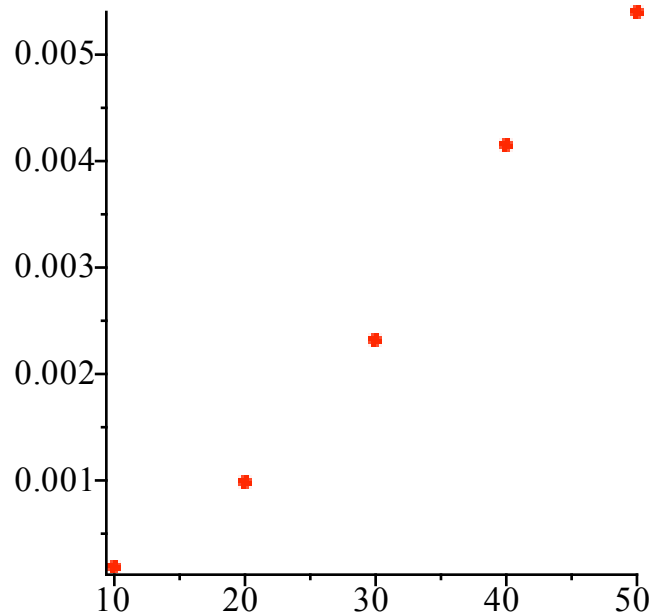
```
> getTimes := proc()
  local sizes, s, avgTimes, times, trials, data, st, t;
  sizes := [seq(10*i, i=1..5)];
  avgTimes := [];
  for s in sizes do
    times := [];
    for trials from 1 to 100 do
      data := combinat[randperm](s);
      st := time();
      bubblesort(data);
      t := time() - st;
      times := [op(times), t];
    end do;
    avgTimes := [op(avgTimes), Statistics[Mean](times)];
  end do;
  return [sizes, avgTimes];
end proc;
```

The procedure begins by generating the list `sizes`. These values indicate the sizes of the lists to which `bubblesort` will be applied. The `avgTimes` list will hold the averages of the times for the corresponding input size. For each possible size, the procedure empties the `times` list and then runs 100 trials in which a randomly ordered list of the appropriate size is generated and the time it takes for `bubblesort` to sort the list is recorded and added to the `times` list. After the 100 trials

are complete, the average of the times is computed and added to the **avgTimes** list. The procedure returns a two-element list consisting of the **sizes** list and the **avgTimes** list. Note that we moved the call to **randperm** to before the assignment of **st**, rather than calling **randperm** within the argument to **bubblesort**. This is so that the time it takes Maple to generate the random permutation is not counted as part of the time it takes **bubblesort** to execute.

Now we apply the **getTimes** procedure and use its output to create a graph.

```
> getTimesOut := getTimes();
getTimesOut := [[10, 20, 30, 40, 50], [0.0001900000000, 0.0009800000000,
0.002320000000, 0.004150000000, 0.005400000000]]
> plot(getTimesOut[1], getTimesOut[2], style=point, symbol=
solidcircle, symbolsize=15);
```

(3.14)


From the shape of the graph, it appears that the average-case performance of **bubblesort** is polynomial. This suggests that the complexity of the algorithm is also polynomial. Of course, a proof of that fact would require an analysis of the kind given in Example 4 of Section 3.3.

The reader can modify the definition of the **sizes** list in order to produce finer detail (by decreasing the step between the input sizes) and to obtain data for larger input lists (by increasing the maximum value of **i**). Note however, that the maximum possible list size that **bubblesort**, as currently written, can handle is 100. For larger sized inputs, you would need to rewrite **bubblesort** and **interchange** to use **Array**s instead of lists as Maple places a limit on the size of a list that you can assign to.

We conclude with a caveat. The empirical testing we've done in this section is an example of a way to get an idea of the average-case performance of a procedure. It can be used to compare two or more algorithms with each other and can indicate major differences in worst-case and average-case complexity (for instance, in an algorithm with exponential worst-case complexity and polynomial average-case complexity). But beyond generalities, the implementation of the algorithm, the computer running it, the computer language it is written in, and a host of other factors can play a sufficiently significant role that this approach is generally not helpful for making finer distinctions

(between quadratic and cubic complexity, for example).

The reader should refer to the solution of Computer Project 9 below for a method of analyzing average case complexity that modifies the procedure in order to count the number of operations used with the input values.

▼ Solutions to Computer Projects and Computations and Explorations

▼ Computer Project 9

Given an ordered list of n integers and an integer x in the list, find the number of comparisons used to determine the position of x in the list using a linear search and using a binary search.

Solution: There is no loss of generality to assume that the list of n integers is the list of integers from 1 to n .

For the linear search algorithm provided as Algorithm 2 in Section 3.1 of the text, each step in the search requires 2 comparisons, one tests whether the end of the list has been reached and one tests whether the current element is the element being searched for. These are both contained in the boolean expression that controls the while loop. A final comparison is used after the while loop is completed to determine whether the element was found or not. In the list of n integers 1 through n , the integer x is therefore found after $2x + 1$ comparisons.

To determine the number of comparisons needed to find x via the binary search algorithm, we'll modify the procedure we wrote in Section 3.1 of this manual to count comparisons. For reference, here is the original **binarysearch** procedure.

```
> binarysearch := proc(x::integer,A::list(integer))
    local n, i, j, m, location;
    n := nops(A);
    i := 1;
    j := n;
    while i < j do
        m := floor((i+j)/2);
        if x > A[m] then
            i := m + 1;
        else
            j := m;
        end if;
    end do;
    if x = A[i] then
        location := i;
    else
        location := 0;
    end if;
    return location;
end proc;
```

We will modify this procedure to count comparisons. Each time through the while loop accounts for 2 comparisons, the $i < j$ comparison that controls the loop and the $x > a_m$ comparison in the if statement. So we'll add a line of code to increment the comparison count by 2 at the start of the while loop. Also, we need to add one to the comparison count after the end of the loop to account for the comparison that terminates the loop. And one final comparison is done to determine if the search has succeeded or not.

We have the procedure return the count of comparisons instead of the position of the element. Here is the modified procedure.

```
> binarysearchC := proc(x::integer,A::list(integer))
    local n, i, j, m, location, count;
    count := 0;
    n := nops(A);
    i := 1;
    j := n;
    while i < j do
        count := count + 2;
        m := floor((i+j)/2);
        if x > A[m] then
            i := m + 1;
        else
            j := m;
        end if;
    end do;
    count := count + 1;
    if x = A[i] then
        location := i;
    else
        location := 0;
    end if;
    count := count + 1;
    return count;
end proc;
```

For example, to find 15 in the list from 1 to 20, it takes

```
> binarysearchC(15, [$1..20]);
                                10
(3.15)
```

comparisons.

We can use the information above to compare the average number of comparisons required in a list of n elements. We need to determine the number of comparisons needed to find each value from 1 to n in the list from 1 to n and average these numbers of comparisons. For the linear search, we know that it takes $2x + 1$ comparisons, so the average can be found from

$$\frac{\sum_{x=1}^n 2x + 1}{n}.$$

We can use Maple's symbolic summation capabilities (discussed in Section 2.4 of this manual).

```
> sum(2*x+1,x=1..n)/n;
                                (n + 1)2 - 1
                                n
(3.16)
```

```
> simplify(%);
                                n + 2
(3.17)
```

(The simplify command forces Maple to simplify expressions.)

For the binary search procedure, we can find the average by applying our procedure above to each integer in turn and taking the average. The following procedure will produce the average number of comparisons required for a given value of n .

```

> binaryAvg := proc(n::posint)
  local comps, L, x;
  comps := [];
  L := [$1..n];
  for x from 1 to n do
    comps := [op(comps), binarysearchC(x, L)];
  end do;
  return Statistics[Mean](comps);
end proc:

```

For example, in the list from 1 to 20, it requires an average of $20 + 2 = 22$ comparisons using the linear search, and an average of 10.8 comparisons using the binary search.

```

> binaryAvg(20);
10.80000000
(3.18)

```

Next, we'll graph the average number of comparisons as n ranges from 1 to 100. To do this, we'll first create the necessary inputs to the `plot` command. Recall from Section 3.2 of this manual that plot requires a list of the x values and a list of the y values. The x values will be the values of n .

```

> nList := [$1..100]:

```

For the linear search algorithm the y values are $n + 2$.

```

> linearAverages := [seq(n+2, n=1..100)]:

```

For the binary search algorithm, the y values are obtained from the procedure `binaryAvg`.

```

> binaryAverages := [seq(binaryAvg(n), n=1..100)]:

```

We want to overlay the graphs for the two algorithms. To do this, we'll assign the results of the `plot` command to a name instead of displaying them. We use different colors for the plots and ensure that the views are the same.

```

> linearPlot := plot(nList, linearAverages, color=blue, view=[1.
  .100, 1..102], legend="linear");
> binaryPlot := plot(nList, binaryAverages, color=red, view=[1.
  .100, 1..102], legend="binary");

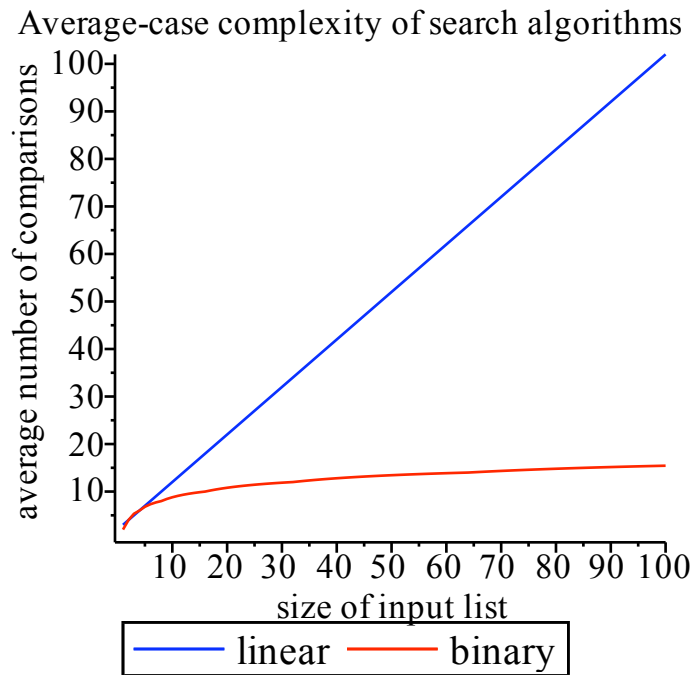
```

To overlay the two plots, we use the `display` command, which is part of the `plots` package. In its simplest form, this command accepts a list of plot structures (such as what you obtain by applying the `plot` command) and overlays the plots. It can also accept most of the options that are available for `plot`. Below, we demonstrate the `display` command along with the use of several options to provide an informative graph.

```

> plots[display]([linearPlot, binaryPlot], title="Average-case
  complexity of search algorithms", labels=["size of input
  list", "average number of comparisons"], labeldirections=
  [horizontal, vertical]);

```

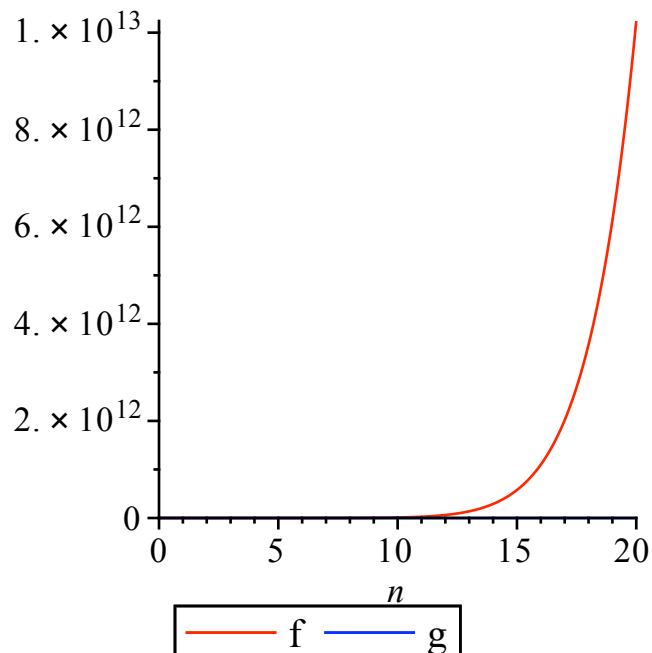


▼ Computations and Explorations 1

We know that n^b is $O(d^n)$ when b and d are positive numbers with $d \geq 2$. Give values of the constants C and k such that $n^b \leq Cd^n$ whenever $n > k$ for each of these sets of values: $b = 10, d = 2$; $b = 20, d = 3$; $b = 1000, d = 7$.

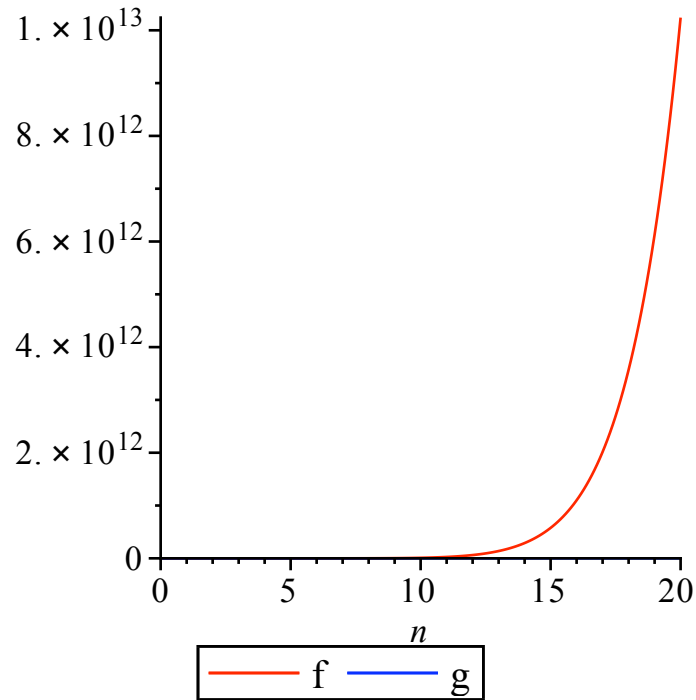
Solution: For $b = 10$ and $d = 2$ we need to compare the functions $f(n) = n^{10}$ to $g(n) = 2^n$. Following the approach we took in Section 3.2, we graph the pair.

```
> plot([n^10, 2^n], n=0..20, color=[red, blue], legend=["f", "g"]);
```

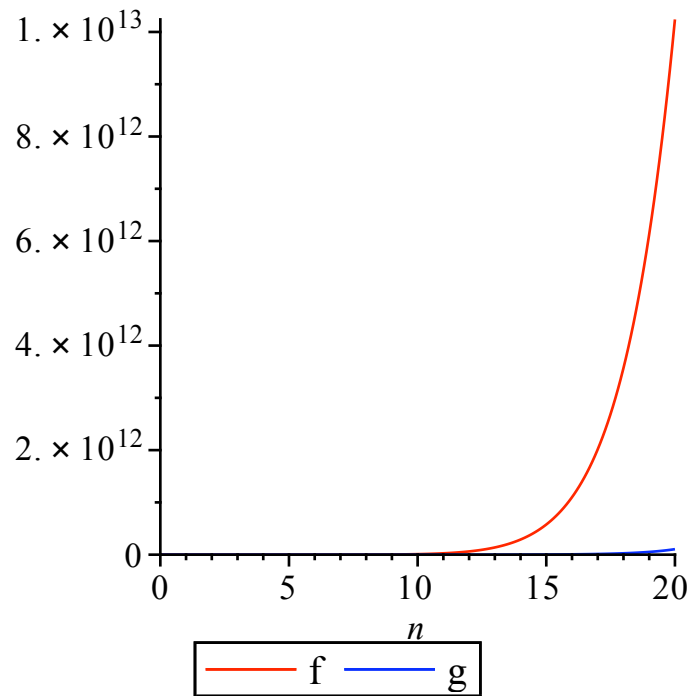


Now, we experiment with different values of C .

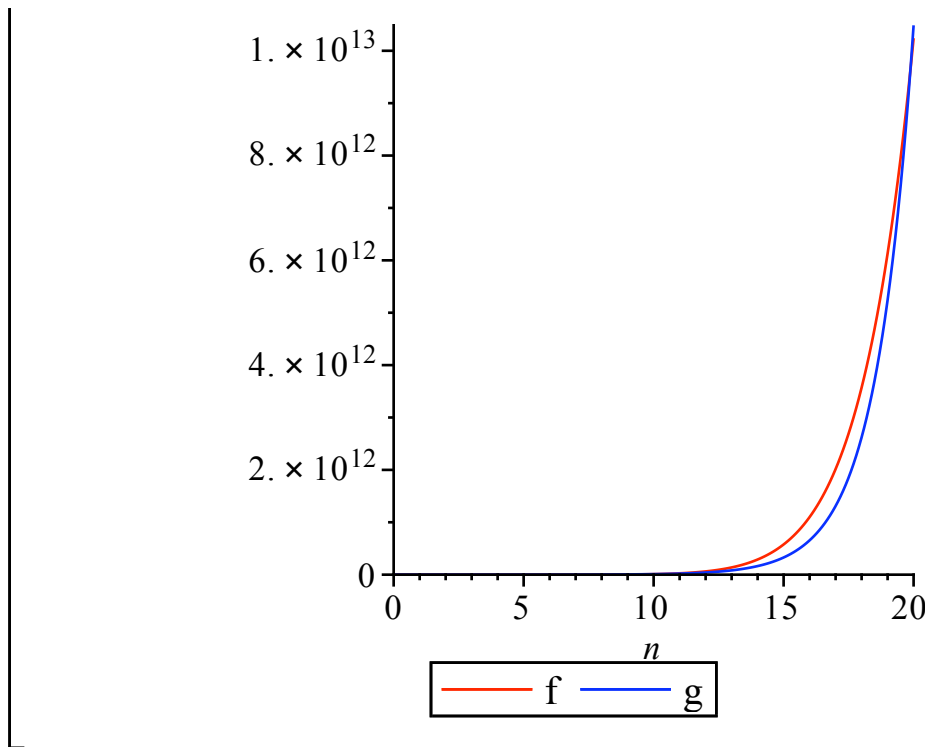
```
> plot([n^10, 10*2^n], n=0..20, color=[red, blue], legend=["f",  
"g"]);
```



```
> plot([n^10, 10^5*2^n], n=0..20, color=[red, blue], legend=["f",  
"g"]);
```



```
> plot([n^10, 10^7*2^n], n=0..20, color=[red, blue], legend=["f",  
"g"]);
```



It appears that with $C = 10^7$, $g(n) = C2^n$ dominates $f(n) = n^{10}$ for $n > 20$. We can confirm this by having Maple solve the inequality $n^{10} \leq 10^7 \cdot 2^n$ with the **solve** command. The **solve** command applied to an equation or inequality in one variable will find the solution to the equation or inequality. We apply **evalf** to force a floating-point result.

```
> evalf(solve(n^10 <= 10^7 * 2^n));
      RealRange(-3.840511598, 10.07828296), RealRange(19.87621431, ∞)      (3.19)
```

This indicates that $C = 10^7$ and $k = 19.9$ witness for n^{10} being $O(2^n)$. Also note that smaller values of C will work provided the value of k is made sufficiently large.

```
> evalf(solve(n^10 <= 10^5 * 2^n));
      RealRange(-2.634476634, 4.243747406), RealRange(34.45805187, ∞)      (3.20)
```

```
> evalf(solve(n^10 <= 10^3 * 2^n));
      RealRange(-1.765447237, 2.347895073), RealRange(44.93042315, ∞)      (3.21)
```

```
> evalf(solve(n^10 <= 1 * 2^n));
      RealRange(-0.9371092012, 1.077550150), RealRange(58.77010593, ∞)      (3.22)
```

▼ Exercises

Exercise 1. Write step-by-step instructions, then pseudocode, and then implement in Maple an algorithm to determine the k largest integers in a list of integers.

Exercise 2. Implement the linear search presented as Algorithm 2 in Section 3.1 of the text.

Exercise 3. Implement the insertion sort presented as Algorithm 5 in Section 3.1 of the text.

Exercise 4. Implement the greedy change-making algorithm presented as Algorithm 6 in Section 3.1 of the text.

Exercise 5. Implement the algorithm for scheduling talks presented as Algorithm 7 in Section 3.1

of the text.

Exercise 6. Implement the brute-force algorithm for finding the closest pair of points as presented in Algorithm 3 in Section 3.3 of the text.

Exercise 7. Modify the **bubblesort** procedure so that it terminates when no more interchanges are necessary. (See Exercise 37 from Section 3.1.)

Exercise 8. Implement the selection sort algorithm in Maple. (Refer to the preamble to Exercise 41 in Section 3.1 for information on selection sort.)

Exercise 9. Implement the binary insertion sort in Maple. (Refer to the preamble to Exercise 47 in Section 3.1 for information on the binary insertion sort.)

Exercise 10. Implement the deferred acceptance algorithm in Maple. (Refer to the preamble to Exercise 61 in Section 3.1 for information on the deferred acceptance algorithm.)

Exercise 11. Following the solution to Computations and Explorations 1, use Maple to determine values for C and k that witness for the fact that $f(x)$ is $O(g(x))$ for each of the pairs of functions given below.

a) $f(x) = 7 \ln(3x^2 - 2x + 5)$; $g(x) = x$.

b) $f(x) = \frac{x^4}{x^2 - 4x - 4}$; $g(x) = x^2$.

c) $f(x) = \lfloor x \rfloor \cdot \lceil x \rceil$; $g(x) = x^2$.

d) $f(x) = n \cdot \ln(n)$; $g(x) = \ln(n!)$.

Exercise 12. Using the approach described in Section 3.3 of this manual, compare the average-case performance of the **bubblesort** procedure presented in Section 3.1 to Maple's **sort** command.

Exercise 13. Using the solution to Computer Project 9 as a model, compare the average-case performance of the **bubblesort** procedure presented in Section 3.1 of this manual with the modified procedure that you implemented as Exercise 7.

Exercise 14. Using the solution to Computer Project 9 as a model, compare the average-case performance of the **bubblesort** procedure with the other sort procedures you wrote (e.g., insertion sort, selection sort, or binary selection sort).